

DATA 260 Homework 4

Database-backed vulnerability reports, an N+1 query comparison, indexing, and a small RAG experiment

Student	Vrishin Dharmesh Kunnatham Parambath
SID4 / Seed	8844
Domain	Open-source package vulnerabilities
Environment	Windows 11, Python 3.14, MySQL 8.0, CPU
Model	Ollama qwen3:8b

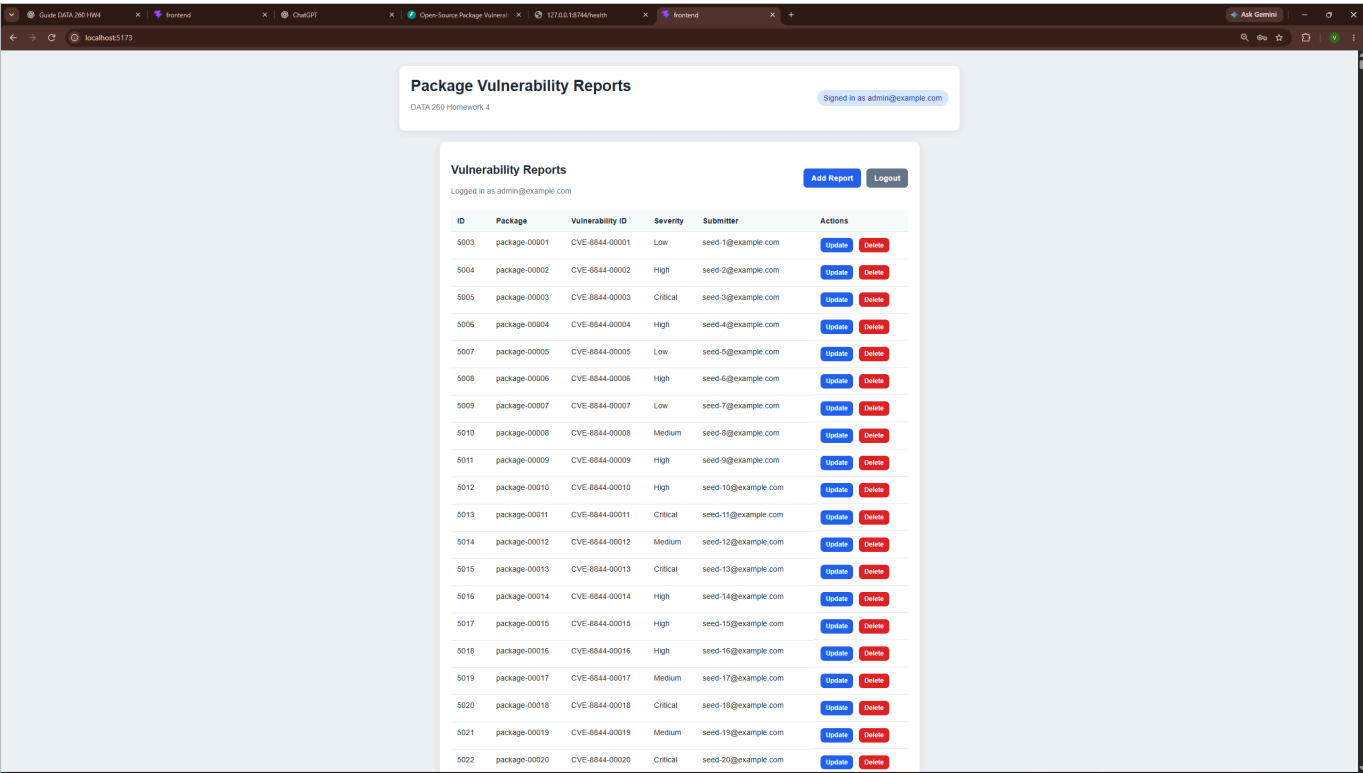
1. What I built

For this homework I extended the vulnerability-report application with a MySQL-backed API and a React frontend. The API uses login protection, sessions, and CRUD operations for reports. I also added benchmark routes so I could compare a naive loading pattern with a fixed version that avoids repeated database queries.

The second part was a small retrieval-augmented generation experiment. I used the documents selected in HW3, split them into chunks, retrieved relevant chunks for six questions, and compared No-RAG, Basic-RAG, and Context-RAG behavior.

2. Database and API checks

The database was seeded with 5,000 vulnerability reports and 200 references. Unauthenticated report access returned HTTP 401, while login and authenticated CRUD requests worked. The screenshot below shows the authenticated dashboard and seeded records.



3. N+1 benchmark

The benchmark used two versions, page sizes of 10, 50, and 200, and 30 requests for each combination. This produced 180 requests in total. The naive version performs repeated lookups while building the response. The fixed version loads related information more efficiently, so the number of SQL statements does not grow in the same way.

```
Windows PowerShell
PS C:\Projects\data260-8844> $table | Format-Table -AutoSize

Version PageSize Requests  SQL  P50_ms  P95_ms  P99_ms
-----
naive    10          30    13.0   17.938   22.52   22.853
naive    50          30    53.0   55.535   61.31   64.368
naive    200         30   203.0  183.201  239.929  284.395
fixed    10          30     4.0    9.648   11.447   11.977
fixed    50          30     4.0   13.567   17.384   18.002
fixed    200         30     4.0   21.786   26.535   45.902

PS C:\Projects\data260-8844>
```

4. Index and EXPLAIN comparison

I compared the package-name query before and after adding an index. Before the index, MySQL estimated a much larger scan. After the index, the access type changed to an indexed reference lookup and the estimated rows dropped to about one for the tested lookup. This shows why indexing a column used in a filter matters even when the query itself is simple.

```
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> Write-Host "===== BEFORE INDEX ====="
===== BEFORE INDEX =====
PS C:\Projects\data260-8844> Get-Content ".\reports\hw04\raw\index_explain_before.txt"
id      select_type  table      partitions  type  possible_keys  key  key_len  ref  rows  filtered  E
-----
extra
1      SIMPLE  vulnerability_reports  NULL  ALL  NULL  NULL  NULL  4885  10.00  Using where
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> Write-Host "`n===== AFTER INDEX ====="
===== AFTER INDEX =====
PS C:\Projects\data260-8844> Get-Content ".\reports\hw04\raw\index_explain_after.txt"
id      select_type  table      partitions  type  possible_keys  key  key_len  ref  rows  filtered  E
-----
extra
1      SIMPLE  vulnerability_reports  NULL  ref  idx_vulnerability_reports_package_name  idx_vulnerability_report
s_package_name  1022  const  1  100.00  NULL
PS C:\Projects\data260-8844> |
```

5. RAG experiment

The corpus contained 69 documents and produced 850 chunks with 500-token chunks and 50-token overlap. I evaluated six questions across three configurations, giving 18 comparison rows. The raw evaluation table is heuristic, so I considered it together with the retrieved chunks and generated answers.

Questions 1 and 2 were answered correctly by the retrieval configurations. Question 3 showed a retrieval limitation because the relevant Pydantic AI source was not in the top three chunks. Question 4 was ambiguous. Questions 5 and 6 were correctly refused by Context-RAG, while No-RAG sometimes produced unsupported answers. Increasing k also brought in unrelated documents for Question 1, which shows that more context is not automatically better.

```
Windows PowerShell
PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> Write-Host "===== RAG EVALUATION ====="
===== RAG EVALUATION =====
PS C:\Projects\data260-8844> $ragTable | Format-Table -AutoSize

question_id configuration correct_retrieval correct_answer_heuristic grounded refused_when_needed
-----
q1          No-RAG          True          False          False          False
q1          Basic-RAG       True          True           True           False
q1          Context-RAG     True          True           True           True
q2          No-RAG          True          False          False          False
q2          Basic-RAG       True          True           True           False
q2          Context-RAG     True          True           True           True
q3          No-RAG          False         True           True           False
q3          Basic-RAG       False         True           True           False
q3          Context-RAG     False         True           True           True
q4          No-RAG          False         False          False          False
q4          Basic-RAG       False         False          False          False
q4          Context-RAG     False         False          False          False
q5          No-RAG          False         False          False          False
q5          Basic-RAG       False         False          False          False
q5          Context-RAG     True          True           True           True
q6          No-RAG          False         False          False          False
q6          Basic-RAG       False         False          False          False
q6          Context-RAG     True          True           True           True

PS C:\Projects\data260-8844>
PS C:\Projects\data260-8844> Write-Host "`n===== K SWEEP ====="
===== K SWEEP =====
PS C:\Projects\data260-8844> Get-Content ".\reports\hw04\raw\k_sweep.json"
===== K SWEEP =====
PS C:\Projects\data260-8844> Get-Content ".\reports\hw04\raw\k_sweep.json"
[
  {
    "question_id": "q1",
    "top_k": 1,
    "sources": [
      "GHS-2599-h6xx-hpxp.json"
    ],
    "chunk_ids": [
      209
    ],
    "scores": [
      0.49261
    ]
  },
  {
    "question_id": "q1",
    "top_k": 3,
    "sources": [
      "GHS-2599-h6xx-hpxp.json",
      "GHS-2599-h6xx-hpxp.json",
      "GHS-24x4-6qmh-88qg.json"
    ],
    "chunk_ids": [
      209,
      213,
      179
    ],
    "scores": [
      0.49261,
      0.362578,
      0.300262
    ]
  },
  {
    "question_id": "q1",
    "top_k": 5,
    "sources": [
      "GHS-2599-h6xx-hpxp.json",
      "GHS-2599-h6xx-hpxp.json",
      "GHS-24x4-6qmh-88qg.json",
      "GHS-247x-2f9f-5wp7.json",
      "GHS-24x6-8c7m-hv3f.json"
    ],
    "chunk_ids": [
      209,
      213,
      179,
      142,
      194
    ],
    "scores": [
      0.49261,
      0.362578,
      0.300262,
      0.296691,
      0.294846
    ]
  }
]
PS C:\Projects\data260-8844>
```

6. Reflection and conclusion

The main result of the database work was that authentication and CRUD behavior worked together with the seeded MySQL data. The benchmark made the N+1 issue visible instead of treating it as only a code-style concern. The index comparison showed a similar point: a small schema change can change how MySQL accesses the table.

The RAG results were mixed, which is expected for a small experiment. Retrieval helped when the right document was returned, but it could not answer a question reliably when the source was missing from the retrieved context. The refusal behavior was useful because it reduced unsupported answers. If I continued the experiment, I would improve document metadata, use a stronger retrieval evaluation set, and add clearer handling for ambiguous questions.

7. Reproducibility files

The repository includes the source code, questions, raw benchmark and RAG outputs, screenshots, METRICS.md, RUN_LOG.txt, AI_USE.md, and verification.json. The verification script completed with **Overall verification: True**.

The measurements in this report came from my local Windows environment. They should be interpreted as results from this run rather than universal performance guarantees.

AI assistance was used for technical tutoring, debugging, and organizing the report. The commands were run locally and the reported measurements came from the saved project outputs.